

Informe técnico comparativo

Repositorio IS-LAB-EIC-UCN/EvaluacionII-I-2025

Patrones de Software y Programación

23 de junio de 2026

Resumen

Este informe analiza la evolución del repositorio EvaluacionII-I-2025, comparando el primer commit, correspondiente al estado inicial sin solución completa, con el último commit disponible, titulado *algunos arreglos*. El análisis se centra en la implementación de la arquitectura Pipe-Filter, el uso de JPA para persistencia, las reglas arquitecturales con ArchUnit, las pruebas unitarias con JUnit 4 y Mockito, y la solución reactiva basada en Vert.x Publish/Subscribe.

Índice

1. Alcance de la comparación	3
2. Problema planteado por la evaluación	3
3. Estado inicial del repositorio	4
3.1. Problemas principales del estado inicial	4
3.1.1. Repositorio incompleto	4
3.1.2. Pipeline no funcional	4
3.1.3. Ausencia de verificación arquitectural	5
4. Reorganización arquitectural en la solución final	5
5. Modelo de dominio y persistencia	6
5.1. RawData	6
5.2. CleanData	6
5.3. Repositorios	6

6. Implementación Pipe–Filter tradicional	7
6.1. Interfaz común de filtros	7
6.2. Servicio orquestador	7
6.3. Cadena final de filtros	8
7. Filtros implementados	8
7.1. ValidatorFilter	8
7.2. UnitNormalizerFilter	9
7.3. ExtremeValueFilter	9
8. Significado del último commit: <i>algunos arreglos</i>	10
9. Reglas ArchUnit implementadas	10
9.1. Los filtros no deben depender de repositorios	10
9.2. Todo filtro debe implementar RawDataFilter	11
10.Pruebas con JUnit 4 y Mockito	11
10.1.Uso de Mockito	12
10.2.Evaluación de calidad de las pruebas	12
11.Implementación reactiva con Vert.x	12
11.1.Componentes principales	13
11.2.Flujo por EventBus	13
11.3.Relación con Publish/Subscribe	13
12.Comparación sintética	14
13.Evaluación crítica	15
13.1.Fortalezas	15
13.2.Debilidades o riesgos	16
14.Conclusión general	16
Referencias	16

1 Alcance de la comparación

El repositorio analizado corresponde a:

<https://github.com/IS-LAB-EIC-UCN/EvaluacionII-I-2025>

La comparación se realiza entre dos estados:

- **First commit:** 8320cb4, titulado *first commit*.
- **Último commit:** 3f2e92a, titulado *algunos arreglos*.

El análisis considera la diferencia publicada entre ambos commits:

<https://github.com/IS-LAB-EIC-UCN/EvaluacionII-I-2025/compare/8320cb4...3f2e92a>

Se trata de un análisis estático del código fuente y del historial del repositorio. No se afirma haber ejecutado localmente la suite de pruebas.

2 Problema planteado por la evaluación

El proyecto plantea un sistema de monitoreo ambiental basado en lecturas de sensores. Dichas lecturas pueden presentar inconsistencias, errores de unidad, valores extremos y problemas de calidad de datos.

La solución esperada combina varios contenidos de la segunda unidad del curso:

1. Arquitectura **Pipe-Filter**.
2. Persistencia mediante **SQLite/JPA**.
3. Reglas arquitecturales automatizadas con **ArchUnit**.
4. Pruebas con **JUnit 4** y **Mockito**.
5. Solución reactiva con **Vert.x** usando **Publish/Subscribe**.

Desde el punto de vista arquitectural, el objetivo principal era desacoplar el procesamiento de datos en etapas independientes, verificables y reemplazables.

3 Estado inicial del repositorio

El primer commit corresponde a un estado incompleto del sistema. El código estaba principalmente concentrado en el paquete `adhoc`, lo que indica una organización inicial poco explícita respecto de responsabilidades arquitecturales.

3.1 Problemas principales del estado inicial

En el estado inicial se observan tres problemas estructurales relevantes.

3.1.1 Repositorio incompleto

El repositorio de datos crudos contenía un método `findAll()` pendiente de implementación. En lugar de consultar la base de datos, retornaba `null`. Esto impedía que el sistema pudiera recuperar lecturas reales para procesarlas.

Listing 1: Estado conceptual inicial de `RawDataRepository`

```
1 public List<RawData> findAll() {  
2     // TODO  
3     return null;  
4 }
```

Este punto era crítico porque el pipeline dependía directamente de la colección de datos entregada por el repositorio.

3.1.2 Pipeline no funcional

El punto de entrada inicial construía una lista de filtros con un valor nulo:

Listing 2: Cadena de filtros incompleta

```
1 List<RawDataFilter> filters = List.of(null);
```

Esto implicaba que el servicio de procesamiento no podía operar correctamente. Aunque existiera la intención de usar `Pipe-Filter`, todavía no existía una cadena real de filtros.

3.1.3 Ausencia de verificación arquitectural

En el primer commit no se observaba una solución completa con reglas ArchUnit, pruebas unitarias suficientes ni verificación automática de dependencias entre capas o paquetes.

Conclusión parcial

El first commit representaba un esqueleto inicial: existían algunas entidades e intenciones de diseño, pero no una solución funcional ni verificable.

4 Reorganización arquitectural en la solución final

La solución final reorganiza el código desde el paquete genérico `adhoc` hacia una estructura basada en responsabilidades:

- `cl.ucn.domain`: entidades de dominio y persistencia.
- `cl.ucn.repository`: acceso a datos.
- `cl.ucn.service`: lógica de orquestación.
- `cl.ucn.service.filters`: filtros del pipeline.
- `cl.ucn.ui`: punto de entrada tradicional.
- `cl.ucn.vertx`: solución reactiva con Vert.x.

Esta reorganización permite expresar reglas arquitecturales sobre paquetes. Por ejemplo, ArchUnit puede verificar que los filtros no dependan directamente de repositorios, o que todas las clases ubicadas en el paquete de filtros implementen la interfaz esperada.

Observación técnica

El cambio desde `adhoc` hacia `cl.ucn.*` no es meramente nominal. La nueva estructura hace visible la arquitectura y permite que esta sea controlada mediante pruebas automatizadas.

5 Modelo de dominio y persistencia

5.1 RawData

La clase RawData representa una lectura cruda de sensor. Contiene información como:

- identificador;
- tipo de medición;
- timestamp;
- valor medido;
- unidad original.

Su propósito es representar datos todavía no depurados. Por tanto, una lectura RawData puede contener unidades no normalizadas o valores que luego serán rechazados por filtros posteriores.

5.2 CleanData

La clase CleanData representa una lectura limpia, es decir, un dato que ya pasó por el proceso de validación, normalización y descarte de valores extremos.

A diferencia de RawData, esta entidad conserva únicamente los datos necesarios para el almacenamiento final. La unidad no aparece como elemento central, lo que sugiere que los valores limpios se asumen normalizados.

5.3 Repositorios

El cambio más importante en persistencia ocurre en RawDataRepository. En el estado final, `findAll()` deja de retornar null y pasa a realizar una consulta JPA:

Listing 3: Consulta conceptual final de RawDataRepository

```
1 return entityManager
2   .createQuery("SELECT r FROM RawData r", RawData.class)
3   .getResultList();
```

Además, el método `close()` queda implementado para cerrar correctamente el `EntityManagerFactory`.

Conclusión parcial

La solución final transforma la persistencia desde un componente incompleto hacia una pieza funcional del flujo de procesamiento.

6 Implementación Pipe–Filter tradicional

6.1 Interfaz común de filtros

La solución define la interfaz `RawDataFilter`. Su contrato puede entenderse de la siguiente forma:

Listing 4: Contrato conceptual de `RawDataFilter`

```
1 public interface RawDataFilter {  
2     RawData apply(RawData data) throws Exception;  
3 }
```

Cada filtro recibe una lectura cruda, puede validarla o transformarla, y luego devuelve el dato para que continúe en la siguiente etapa. Si el dato no cumple una condición, el filtro puede lanzar una excepción.

6.2 Servicio orquestador

El servicio `RawDataProcessingService` cumple el rol de orquestador del pipeline. Su responsabilidad no es aplicar una regla específica, sino coordinar el flujo completo:

1. obtener todas las lecturas crudas;
2. aplicar secuencialmente los filtros;
3. construir una entidad `CleanData`;

4. persistir el dato limpio;
5. descartar datos inválidos si algún filtro lanza una excepción.

El diseño es coherente con Pipe-Filter porque cada etapa mantiene una responsabilidad acotada.

6.3 Cadena final de filtros

En el punto de entrada tradicional, la cadena queda configurada como:

Listing 5: Pipeline tradicional final

```
1 List<RawDataFilter> filters = List.of(  
2     new ValidatorFilter(),  
3     new UnitNormalizerFilter(),  
4     new ExtremeValueFilter()  
5 );
```

El orden es técnicamente correcto:

- primero se valida que el dato tenga estructura mínima;
- luego se normalizan unidades;
- finalmente se evalúan rangos extremos sobre datos ya normalizados.

7 Filtros implementados

7.1 ValidatorFilter

El filtro `ValidatorFilter` valida condiciones estructurales mínimas:

- el objeto no debe ser nulo;
- el tipo no debe ser nulo;
- el timestamp no debe ser nulo;
- el valor no debe ser considerado inválido;

- el tipo debe ser `temperature` o `mp`.

Si alguna condición falla, lanza una excepción de tipo `IllegalArgumentException`.

Observación técnica

Existe un punto discutible: el filtro trata el valor 0 como inválido. En un dominio ambiental, 0 puede ser perfectamente válido, por ejemplo 0 °C o 0 ug/m3. Por tanto, esta regla debería revisarse.

7.2 UnitNormalizerFilter

El filtro `UnitNormalizerFilter` transforma unidades no estándar.

Para temperatura, si la unidad es Fahrenheit, aplica:

$$C = \frac{F - 32}{1,8}$$

Para material particulado, si la unidad es mg/m3, transforma a ug/m3 mediante:

$$ug/m^3 = mg/m^3 \times 1000$$

Este filtro no descarta datos: transforma su representación para que las etapas posteriores trabajen sobre una unidad común.

7.3 ExtremeValueFilter

El filtro `ExtremeValueFilter` rechaza valores fuera de rangos aceptables:

- temperatura válida: entre -50 y 70;
- material particulado válido: entre 0 y 1000.

Si el dato excede esos rangos, se lanza una excepción. Si no, el dato continúa.

Conclusión parcial

La separación entre `ValidatorFilter`, `UnitNormalizerFilter` y `ExtremeValueFilter` es adecuada: cada filtro tiene una responsabilidad clara dentro del pipeline.

8 Significado del último commit: *algunos arreglos*

El último commit modifica principalmente `ValidatorFilter`. Antes de ese cambio, el filtro de validación mezclaba dos responsabilidades:

1. validar estructura del dato;
2. validar rangos de temperatura.

En el último commit, la validación de rangos se elimina desde `ValidatorFilter`, dejando esa responsabilidad en `ExtremeValueFilter`.

Esto mejora la cohesión del diseño. En una arquitectura Pipe-Filter, cada filtro debe tener una responsabilidad clara. Si `ValidatorFilter` rechaza valores extremos, entonces invade la responsabilidad del filtro de extremos.

Observación técnica

Aunque el commit se llame *algunos arreglos*, el cambio es arquitectónicamente importante: corrige la asignación de responsabilidades dentro del pipeline.

9 Reglas ArchUnit implementadas

La solución agrega una clase de pruebas arquitecturales. Conceptualmente, las reglas implementadas verifican dos decisiones.

9.1 Los filtros no deben depender de repositorios

La primera regla impide que clases ubicadas en paquetes de filtros dependan de clases ubicadas en paquetes de repositorio.

Listing 6: Regla conceptual: filtros no acceden a repositorios

```
1 noClasses()  
2   .that().resideInAPackage("..filters..")  
3   .should().dependOnClassesThat()  
4   .resideInAPackage("..repository..");
```

Esta regla protege la arquitectura Pipe-Filter. Los filtros deben procesar datos, no consultar ni guardar información directamente en la base de datos.

9.2 Todo filtro debe implementar RawDataFilter

La segunda regla exige que las clases concretas del paquete de filtros implementen el contrato común:

Listing 7: Regla conceptual: filtros implementan RawDataFilter

```
1 classes()  
2   .that().resideInAPackage("..filters..")  
3   .should().implement(RawDataFilter.class);
```

Esta regla protege la uniformidad del pipeline: todo nuevo filtro debe poder integrarse a la misma cadena de procesamiento.

Observación técnica

El enunciado permitía que el último filtro pudiera acceder a base de datos. La solución final adopta una restricción más estricta: ningún filtro accede al repositorio. Esta decisión es defendible porque el almacenamiento se delega al servicio y al repositorio, no a un filtro.

10 Pruebas con JUnit 4 y Mockito

La solución agrega pruebas para los componentes principales. Se observan pruebas para:

- validación de datos;
- normalización de unidades;

- detección de valores extremos;
- persistencia de datos limpios;
- procesamiento del servicio usando Mockito.

10.1 Uso de Mockito

La prueba más importante desde el punto de vista de aislamiento es la del servicio de procesamiento. Allí se simula el repositorio de datos crudos:

Listing 8: Uso conceptual de Mockito

```
1 RawDataRepository rawRepo = Mockito.mock(RawDataRepository.class);  
2  
3 when(rawRepo.findAll()).thenReturn(List.of(  
4     new RawData(...)  
5 ));
```

Esto permite probar el servicio sin depender de una base de datos real. Así se cumple el principio de prueba unitaria: aislar la unidad bajo prueba y controlar sus dependencias.

10.2 Evaluación de calidad de las pruebas

La cobertura conceptual es adecuada para la pauta. Sin embargo, hay dos aspectos mejorables:

1. deberían existir más casos de borde, especialmente para valor 0;
2. las pruebas de repositorio se acercan más a pruebas de integración que a pruebas unitarias puras.

11 Implementación reactiva con Vert.x

La solución incorpora una variante reactiva basada en Vert.x y el patrón Publish/Subscribe.

11.1 Componentes principales

El paquete `cl.ucn.vertx` incluye verticles para las distintas etapas del flujo:

- `ReaderBDVerticle`;
- `ProducerBDVerticle`;
- `ValidatorFilterVerticle`;
- `UnitNormalizerFilterVerticle`;
- `ExtremeValueFilterVerticle`;
- `FileStorageVerticle`;
- `MainVerticle`.

11.2 Flujo por EventBus

La solución reactiva utiliza direcciones lógicas del `EventBus`. El flujo puede representarse así:

`raw.data.incoming` → `validated.data` → `normalized.data` → `filtered.data`

Cada filtro reactivo consume desde una dirección y publica hacia la siguiente. Esto evita llamadas directas entre clases y reduce el acoplamiento entre componentes.

11.3 Relación con Publish/Subscribe

El patrón `Publish/Subscribe` se evidencia en el uso de `publish()` para difundir eventos hacia una dirección del `EventBus`. Los consumidores no conocen directamente al emisor; solo conocen la dirección a la que están suscritos.

Esto cumple con la idea central de la programación reactiva: comunicación asíncrona, orientada a eventos y con bajo acoplamiento temporal y espacial.

Observación técnica

Debe revisarse que `FileStorageVerticle` escuche el canal correcto. Si sigue escuchando `validated.data`, podría almacenar datos antes de que pasen por normalización y filtro de extremos. El canal final más coherente sería `filtered.data`.

12 Comparación sintética

Aspecto	First commit	Último commit	Efecto técnico
Paquetes	Código concentrado en <code>adhoc</code> .	Código reorganizado en paquetes <code>cl.ucn.*</code> .	Mejora modularidad y verificabilidad.
Repositorio crudo	<code>findAll()</code> retornaba <code>null</code> .	<code>findAll()</code> consulta datos con JPA.	Permite alimentar el pipeline.
Cadena de filtros	<code>List.of(null)</code> .	Tres filtros concretos: validación, normalización y extremos.	El pipeline pasa a ser funcional.
Validación	No había filtro real completo.	Se valida estructura y tipo de dato.	Se descartan datos inválidos.
Normalización	No implementada.	Conversión de Fahrenheit a Celsius y de <code>mg/m3</code> a <code>ug/m3</code> .	Unifica unidades antes del almacenamiento.
Valores extremos	No implementado.	Rangos para temperatura y material particulado.	Se eliminan lecturas físicamente sospechosas.
ArchUnit	Sin verificación arquitectural completa.	Reglas para filtros y dependencias.	La arquitectura se vuelve ejecutable.
Tests	Ausentes o insuficientes.	Pruebas con JUnit 4 y Mockito.	Mejora la verificación funcional.
Vert.x	Solución reactiva incompleta.	Filtros como verticles y comunicación por <code>EventBus</code> .	Implementa <code>Publish/Subscribe</code> .

Aspecto	First commit	Último commit	Efecto técnico
Último arreglo	No aplica.	ValidatorFilter deja de validar rangos.	Mejora separación de responsabilidades.

13 Evaluación crítica

La solución final implementa de manera razonablemente completa la parte central de la evaluación. El repositorio pasa de un estado inicial incompleto a una solución con:

- pipeline funcional;
- filtros concretos;
- persistencia mediante repositorios;
- pruebas unitarias;
- reglas arquitecturales;
- variante reactiva con Vert.x.

La mejora más importante no es solamente agregar clases, sino ordenar responsabilidades. Los repositorios acceden a datos; los filtros procesan datos; el servicio orquesta; ArchUnit verifica reglas; y Vert.x modela una variante desacoplada por eventos.

13.1 Fortalezas

1. La cadena `ValidatorFilter ->UnitNormalizerFilter ->ExtremeValueFilter` es coherente.
2. ArchUnit se usa para convertir restricciones arquitecturales en pruebas ejecutables.
3. Mockito permite probar el servicio sin depender de la base de datos real.
4. La solución reactiva respeta el desacoplamiento mediante EventBus.

13.2 Debilidades o riesgos

1. El rechazo del valor 0 en `ValidatorFilter` es discutible.
2. Las pruebas podrían cubrir más casos de borde.
3. Debe verificarse cuidadosamente que el almacenamiento reactivo escuche el canal final del pipeline.
4. Algunas pruebas de persistencia son más cercanas a integración que a unidad.

14 Conclusión general

Comparado con el first commit, el último commit transforma el repositorio desde un estado inicial incompleto hacia una solución evaluable. La implementación final satisface el núcleo de la pauta: Pipe-Filter funcional, filtros concretos, persistencia JPA, pruebas JUnit/Mockito, reglas ArchUnit y una variante Vert.x Publish/Subscribe.

No obstante, todavía existen detalles que un corrector estricto podría observar: la validación excesiva del valor cero, pruebas perfectibles y una posible desalineación de canales en la solución reactiva. Aun así, el salto arquitectural entre el estado inicial y el estado final es claro: el proyecto deja de ser un esqueleto *ad hoc* y pasa a expresar una arquitectura modular, verificable y parcialmente reactiva.

Referencias

- Repositorio principal: <https://github.com/IS-LAB-EIC-UCN/EvaluacionII-I-2025>
- Comparación entre commits: <https://github.com/IS-LAB-EIC-UCN/EvaluacionII-I-2025/compare/8320cb4...3f2e92a>
- Commit inicial: <https://github.com/IS-LAB-EIC-UCN/EvaluacionII-I-2025/commit/8320cb4>
- Último commit analizado: <https://github.com/IS-LAB-EIC-UCN/EvaluacionII-I-2025/commit/3f2e92a>